

Fall 2025 CPTS530 Midterm Project
MATH 448-548
Numerical Analysis

Aryan Ritwajeet Jha

14 October 2025

Contents

Problem 1	2
Problem 2	3
Problem 3	4
Problem 4	8
Problem 4.1	8
Problem 4.2	9
Problem 4.3	10
Problem 4.4	11
Problem 4.5	12
Problem 4.6	13
Problem 5	14
Problem 5.1	14
Problem 5.2	16
Problem 5.3	18
Problem 5.4	20
A Code Implementations	26
A.1 Problem 3: LU and Cholesky Factorization	26
A.2 Problem 5-3: Ridders' Method Function	29

Problem 1

Problem Statement: Use Taylor theorem to derive the error term for the approximate formula

$$f'(x) \approx \frac{1}{2h}(-3f(x) + 4f(x+h) - f(x+2h)).$$

Solution:

We are given the finite-difference formula:

$$f'(x) \approx \frac{1}{2h} [-3f(x) + 4f(x+h) - f(x+2h)]$$

Expand $f(x+h)$ and $f(x+2h)$ about x using Taylor's theorem, keeping terms up to h^2 and representing higher-order terms as k_1h^3 and k_2h^4 :

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + k_1h^3 \\ f(x+2h) &= f(x) + 2hf'(x) + 2h^2f''(x) + k_2h^3 \end{aligned}$$

where k_1, k_2 are finite constants depending on third order derivatives of f at x .

Substitute into the formula:

$$\begin{aligned} &\frac{1}{2h} [-3f(x) + 4f(x+h) - f(x+2h)] \\ &= \frac{1}{2h} \left[-3f(x) + 4 \left(f(x) + hf'(x) + \frac{h^2}{2}f''(x) + k_1h^3 \right) \right. \\ &\quad \left. - (f(x) + 2hf'(x) + 2h^2f''(x) + k_2h^3) \right] \end{aligned}$$

Simplifying the terms in the square brackets:

$$= \frac{1}{2h} [2hf'(x) + (4k_1 - k_2)h^3]$$

Now, divide each term by $2h$:

$$= f'(x) + \frac{1}{2}(4k_1 - k_2)h^2$$

Thus, the leading error term is $O(h^2)$:

$$\text{error} \sim O(h^2)$$

This shows the method is second-order accurate. Note that I've already checked if the third order term vanishes if we write out the Taylor's polynomial to the third order, and it does not.

Problem 2

Problem Statement: Consider the following variation of the Newton's method:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_0)}.$$

Find constants C and s such that

$$e_{n+1} = C e_n^s.$$

Solution:

Problem 3

Problem Statement: Consider the linear system $\mathbf{Ax} = \mathbf{b}$, where

$$\mathbf{A} = \begin{bmatrix} 6.25 & -1 & 0.5 \\ -1 & 5 & 2.12 \\ 0.5 & 2.12 & 3.6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 7.5 \\ -8.68 \\ -0.24 \end{bmatrix}.$$

Write a computer program for LU-factorization with a unit lower triangular $\mathbf{L}$ (meaning that the diagonal entries should be equal to one). Then write a program for the Cholesky factorization.

WARNING: avoid using shortcuts. The programming should be done “from scratch”.

Solution:

Implementation Approach

I implemented both LU (Doolittle) and Cholesky factorizations from scratch in Julia, using simple nested loops without any built-in matrix factorization functions. The implementations follow the standard algorithms.

Method 1: LU Factorization

The LU factorization decomposes $\mathbf{A} = \mathbf{LU}$, where $\mathbf{L}$ is unit lower triangular and $\mathbf{U}$ is upper triangular. Figure 1 shows the complete output including the factorization, verification, and solution.

```

=====
METHOD 1: LU Factorization
=====

L (lower triangular, 1's on diagonal):
3×3 Matrix{Float64}:
 1.0  0.0  0.0
-0.16 1.0  0.0
 0.08 0.454545 1.0

U (upper triangular):
3×3 Matrix{Float64}:
 6.25 -1.0  0.5
 0.0  4.84  2.2
 0.0  0.0  2.56

Verification: max|A - LU| = 4.441e-16

Solution from LU factorization:
x =
3-element Vector{Float64}:
 0.8
-2.0
 1.0
Residual: max|Ax - b| = 2.220e-16
Difference from Julia's A\b: 2.220e-16
✓ LU solution matches built-in solver!

```

Figure 1: LU Factorization output showing the factored matrices $\mathbf{L}$ and $\mathbf{U}$, verification that $\mathbf{A} = \mathbf{LU}$, the computed solution $\mathbf{x}$, and comparison with Julia's built-in solver. The factorization is verified to machine precision ($\sim 10^{-16}$), and the solution matches the built-in solver exactly.

The factorization yields:

$$\mathbf{L} = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ -0.16 & 1.0 & 0.0 \\ 0.08 & 0.454545 & 1.0 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 6.25 & -1.0 & 0.5 \\ 0.0 & 4.84 & 2.2 \\ 0.0 & 0.0 & 2.56 \end{bmatrix}$$

Verification: $\max |\mathbf{A} - \mathbf{LU}| = 4.441 \times 10^{-16}$

Method 2: Cholesky Factorization

The Cholesky factorization exploits the symmetry and positive-definiteness of $\mathbf{A}$, decomposing it as $\mathbf{A} = \mathbf{LL}^T$, where $\mathbf{L}$ is lower triangular. Figure 2 shows the complete output.

```
=====
METHOD 2: Cholesky Factorization
=====

L (Cholesky factor):
3×3 Matrix{Float64}:
 2.5  0.0  0.0
-0.4  2.2  0.0
 0.2  1.0  1.6

Verification: max|A - LL'| = 8.882e-16

Solution from Cholesky factorization:
x =
3-element Vector{Float64}:
 0.8
-1.9999999999999996
 0.9999999999999998
Residual: max|Ax - b| = 2.220e-16
Difference from Julia's A\b: 4.441e-16
✓ Cholesky solution matches built-in solver!

Comparison: max|x_LU - x_Cholesky| = 4.441e-16
✓ Both methods agree perfectly!
```

Figure 2: Cholesky Factorization output showing the Cholesky factor $\mathbf{L}$, verification that $\mathbf{A} = \mathbf{LL}^T$, the computed solution $\mathbf{x}$, and comparison with Julia's built-in solver. The factorization is verified to machine precision, and both LU and Cholesky methods produce identical solutions.

The Cholesky factor is:

$$\mathbf{L}_{\text{Cholesky}} = \begin{bmatrix} 2.5 & 0.0 & 0.0 \\ -0.4 & 2.2 & 0.0 \\ 0.2 & 1.0 & 1.6 \end{bmatrix}$$

Verification: $\max |\mathbf{A} - \mathbf{LL}^T| = 8.882 \times 10^{-16}$

Final Solution

Both methods yield the same solution (verified to machine precision):

$$\mathbf{x} = \begin{bmatrix} 0.8 \\ -2.0 \\ 1.0 \end{bmatrix}$$

Verification and Testing

The implementations were thoroughly verified:

- LU factorization accuracy: $\max |\mathbf{A} - \mathbf{L}\mathbf{U}| = 4.441 \times 10^{-16}$
- Cholesky factorization accuracy: $\max |\mathbf{A} - \mathbf{L}\mathbf{L}^T| = 8.882 \times 10^{-16}$
- Solution residual: $\max |\mathbf{A}\mathbf{x} - \mathbf{b}| = 2.220 \times 10^{-16}$
- Comparison with Julia's built-in solver: $\max |\mathbf{x}_{\text{computed}} - \mathbf{x}_{\text{builtin}}| = 2.220 \times 10^{-16}$
- LU vs Cholesky solution: $\max |\mathbf{x}_{\text{LU}} - \mathbf{x}_{\text{Cholesky}}| = 4.441 \times 10^{-16}$

All errors are at machine precision level ($\sim 10^{-16}$), confirming the correctness of both implementations. The complete Julia codebase can be viewed in Appendix A.1.

Problem 4

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2} \left(\cos(x/2) - \left| x - \frac{1}{2} \right| \right).$$

Do the following.

Part 1

Rewrite the equation as a fixed point problem for a function $T(x)$ related to $f(x)$. Prove that $T(x)$ (i) maps the interval $[0.45, 0.55]$ into itself, and (ii) $T(x)$ is contractive on this interval. Based on the theorems in the books and notes, make conclusions about existence and uniqueness of a solution to $f(x) = 0$.

Solution:

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2} \left(\cos(x/2) - \left| x - \frac{1}{2} \right| \right).$$

Do the following.

Part 2

Write the computer program implementing the fixed point iteration $x_{n+1} = T(x_n)$. Discuss the choice of an initial guess x_0 . Calculate the solution to within 10 decimal places. Note the number of iterations needed.

Solution:

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2} \left(\cos(x/2) - \left| x - \frac{1}{2} \right| \right).$$

Do the following.

Part 3

Use a priori and a posteriori error estimates from the notes to estimate the number of iterations needed to obtain the above accuracy. Which estimate is more accurate?

Solution:

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2} \left(\cos(x/2) - \left| x - \frac{1}{2} \right| \right).$$

Do the following.

Part 4

Implement the Newton's method on a computer for the same equation starting with x_0 within the interval $[0.45, 0.55]$. Discuss the theoretical difficulty related to non-existence of $f'(x)$ for certain point(s) within the interval. How would you set up the Newton's method to resolve the problem?

Solution:

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2} \left(\cos(x/2) - \left| x - \frac{1}{2} \right| \right).$$

Do the following.

Part 5

Note the number of iterations needed to calculate the solution to within 10 decimal places using the Newton's method. Discuss the speed of convergence of the Newton's method and compare it with the convergence speed of the fixed point iterations.

Solution:

Problem Statement: Consider the equation

$$f(x) = 0,$$

where

$$f(x) = x - \frac{1}{2} \left(\cos(x/2) - \left| x - \frac{1}{2} \right| \right).$$

Do the following.

Part 6

Implement the alternative error control strategy proposed by you in HW 3, Problem 2. Compare the results with the ones obtained using a priori and a posteriori error estimates.

Solution:

Problem 5

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

Brief Description of the method

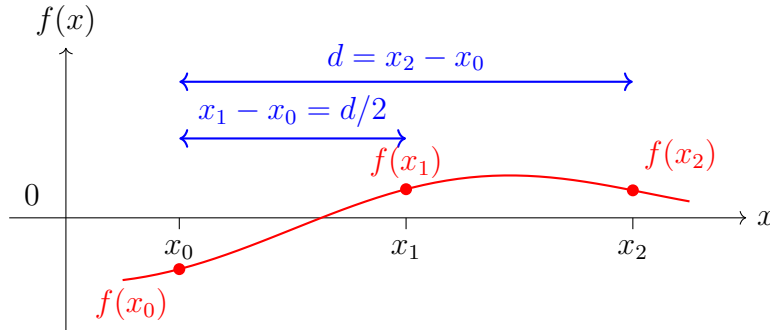
Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x -intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 1

Show that a is uniquely defined and give the equation for finding it.

Solution:

Note: Thanks to the wikipedia article which itself cites the original paper by Ridders, it was easier to derive all the relevant formulae. I have chosen my target formulae and required derivations to align with those, with little variation.



Let x_0 and x_2 be the endpoints of the bracketing interval, and $x_1 = \frac{x_0 + x_2}{2}$ the midpoint. Let $d = x_2 - x_0$.

Define $f_0 = f(x_0)$, $f_1 = f(x_1)$, $f_2 = f(x_2)$.

Ridders' method introduces a function $h(x) = f(x)e^{ax}$, and chooses a so that:

$$h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$$

$$f_1 e^{ax_1} = \frac{1}{2}(f_0 e^{ax_0} + f_2 e^{ax_2})$$

We know that $x_1 = x_0 + \frac{d}{2}$ and $x_2 = x_0 + d$.

Substitute and rearrange:

$$f_1 e^{a(x_0+d/2)} = \frac{1}{2} (f_0 e^{ax_0} + f_2 e^{a(x_0+d)})$$

Dividing both sides by e^{ax_0} :

$$2f_1 e^{ad/2} = f_0 + f_2 e^{ad}$$

$$2f_1 e^{ad/2} - f_2 e^{ad} - f_0 = 0$$

Let $y = e^{ad/2}$, so $e^{ad} = y^2$.

This gives a quadratic in y :

$$f_2 y^2 - 2f_1 y + f_0 = 0$$

Solving for y :

$$y = \frac{2f_1 \pm \sqrt{(2f_1)^2 - 4f_2 f_0}}{2f_2}$$

$$y = \frac{f_1 \pm \sqrt{f_1^2 - f_2 f_0}}{f_2}$$

Therefore,

$$a = \frac{2}{d} \ln \left(\frac{f_1 \pm \sqrt{f_1^2 - f_2 f_0}}{f_2} \right)$$

We note that since $y = e^{ad/2}$, its value must be nonnegative. Since clearly $f_1^2 - f_2 f_0 > f_1^2$ ($f_2 * f_0 < 0$), the sign is chosen so that y is positive:

$$\text{If } f_2 > 0 : \quad a = \frac{2}{d} \ln \left(\frac{f_1 + \sqrt{f_1^2 - f_2 f_0}}{f_2} \right)$$

$$\text{else If } f_2 < 0 : \quad a = \frac{2}{d} \ln \left(\frac{f_1 - \sqrt{f_1^2 - f_2 f_0}}{f_2} \right)$$

Or, more concisely:

$$a = \frac{2}{d} \ln \left(\frac{f_1 + \text{sgn}(f_2) \sqrt{f_1^2 - f_2 f_0}}{f_2} \right)$$

or equivalently:

where $\text{sgn}(f_2)$ is the sign of f_2 .

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

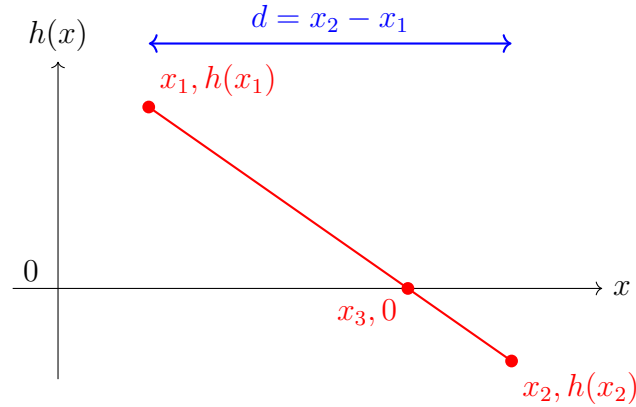
Brief Description of the method

Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x -intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 2

Work out a formula for x_3 in detail.

Solution:



From the diagram, we want to find x_3 , the x -intercept of the line passing through $(x_1, h(x_1))$ and $(x_2, h(x_2))$.

Let $h_1 = h(x_1)$ and $h_2 = h(x_2)$. The slope of the line is:

$$m = \frac{h(x_1) - h(x_2)}{x_1 - x_2} = \frac{h_1 - h_2}{x_1 - x_2}$$

The equation of the line passing through (x_1, h_1) with slope m is:

$$h(x) - h_1 = m(x - x_1)$$

To find x_3 , set $h(x_3) = 0$:

$$0 = h_1 + m(x_3 - x_1)$$

Solving for x_3 :

$$m(x_3 - x_1) = -h_1$$

$$x_3 - x_1 = -\frac{h_1}{m}$$

$$x_3 = x_1 - \frac{h_1}{m}$$

Substitute the expression for m :

$$x_3 = x_1 - \frac{h_1}{\frac{h_1 - h_2}{x_1 - x_2}}$$

$$x_3 = x_1 - \frac{h_1(x_1 - x_2)}{h_1 - h_2}$$

Now, using $d/2 = x_2 - x_1$, we have:

$$x_3 = x_1 + \frac{d}{2} \cdot \frac{1}{1 - h_2/h_1}$$

Or equivalently:

$$x_3 = \frac{x_1 h_2 - x_2 h_1}{h_2 - h_1}$$

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

Brief Description of the method

Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x -intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 3

Implement Ridders' method on a computer together with the standard bisection method for finding **all** solutions of

$$e^x - x^2 = 0.$$

Solution:

First I'll plot the graph of $e^x - x^2$ to identify the intervals where the function changes sign, indicating the presence of roots.

From Figure 3, it can be seen that there is one root in the interval $[-0.8, -0.6]$. Since $f(x) < 0$ for $x = -0.8$ and $f(x) > 0$ for $x = -0.6$, I can use this interval to find the root using both the bisection method and Ridders' method.

Root Finding on the Original Interval

For the purpose of **root finding**, I applied both methods to the smaller interval $[-0.8, -0.6]$ with tolerances $\epsilon = 10^{-5}$ and $\delta = 10^{-5}$. The results are shown in Figure 4.

As shown in Figure 4, Ridders' method found the root $x = -0.703468627463942$ with $|f(x)| \approx 2.29 \times 10^{-6}$ in only **1 iteration**, while the bisection method required **12 iterations** to find $c = -0.703466796875000$ with $|f(c)| \approx 1.19 \times 10^{-6}$. This dramatic difference in iteration count demonstrates the superior efficiency of Ridders' method even on a relatively small interval.

The full code for the Ridders' method function is provided in Appendix A.2. The bisection code is not shown here, as it was implemented in Homework 1 and is a subset of the Ridders' approach.

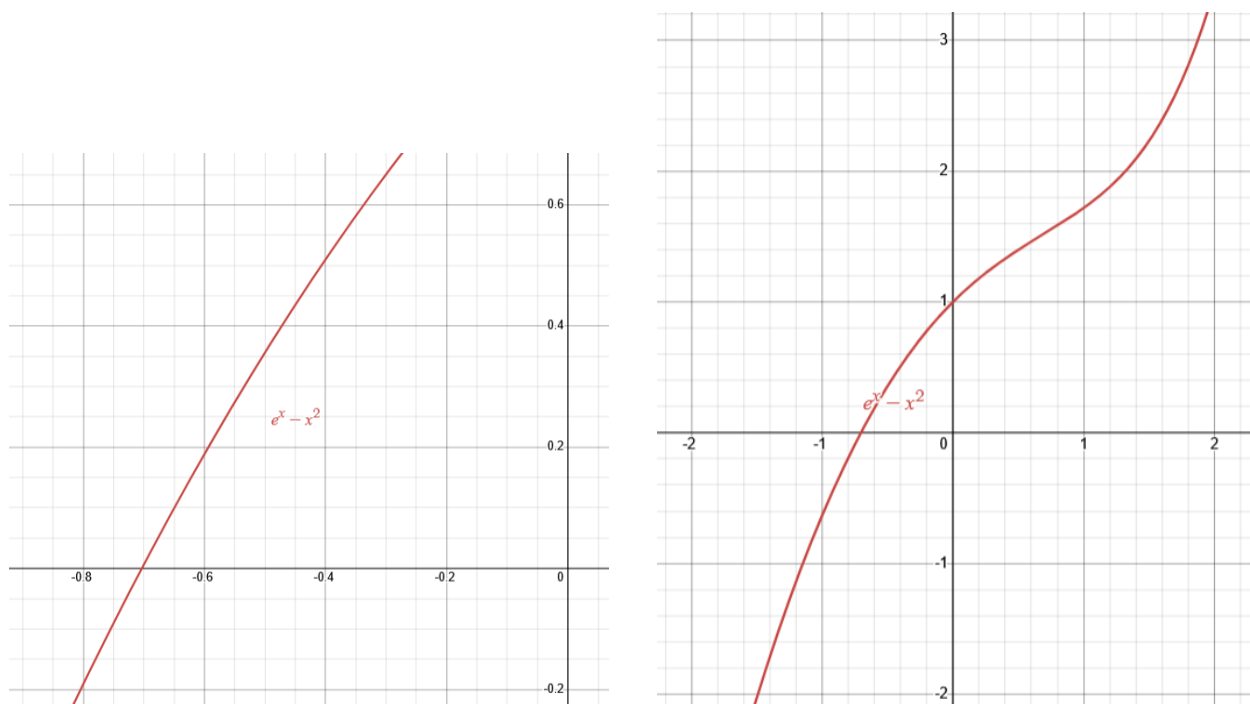


Figure 3: Graphs of $e^x - x^2$: We can see from the zoomed in graph on the left that there is a single root in $[-0.8, -0.6]$. The zoomed out graph on the right shows that the function is monotonically increasing and has no other zeros.

```

=====
PART 1: Original Problem Settings (as given in midterm)
=====
Interval: [-0.8, -0.6],  $\epsilon=1e-5$ ,  $\delta=1e-5$ 

Ridders Method:
Root found at x3 = -0.7034686274639423 with f(x3) = -2.2916060946620576e-6
✓ Ridders converged in 1 iteration(s)
  Root: x = -0.703468627463942
  f(x): -2.292e-06

Bisection Method:
Root found at c = -0.7034667968750001 with f(c) = 1.189811059454371e-6
✓ Bisection converged in 12 iteration(s)
  Root: c = -0.703466796875000
  f(c): 1.190e-06
    
```

Figure 4: Terminal output comparing root finding results for $e^x - x^2$ using both Ridders' method and bisection method on the interval $[-0.8, -0.6]$. Ridders' method converged in just 1 iteration, while bisection required 12 iterations to achieve similar accuracy.

Problem Statement: Implement and analyze the Ridders' method for solving the equation $f(x) = 0$. For guidance, you might wish to read the Wikipedia page on the Ridders' method. Also feel free to consult other sources.

Brief Description of the method

Given the initial bracketing interval $[x_0, x_2]$, containing **one** solution, and such that $f(x_0)$ and $f(x_2)$ have opposite signs, compute the midpoint $x_1 = (x_0 + x_2)/2$. Then define a new function $h(x) = f(x)e^{ax}$. Find the parameter a that ensures $h(x_1) = \frac{1}{2}(h(x_0) + h(x_2))$. Then define x_3 as the x -intercept of the line passing through $(x_0, h(x_0))$ and $(x_2, h(x_2))$. Use x_3 as one of the endpoints of the next interval bracketing the solution. The other endpoint is x_1 if $f(x_1)f(x_3) < 0$. Otherwise, choose either x_0 or x_2 based on the requirement that the sign of $f(x)$ at the chosen point must be opposite to the sign of $f(x_3)$. Continue until the desired accuracy is reached.

Part 4

Numerically, compare the convergence rate of both algorithms. Discuss the results. What is the approximate order of convergence for the Ridders' algorithm?

Solution:

Need for a Wider Interval

The interval $[-0.8, -0.6]$ from Part 3 is too small for convergence analysis—Ridders' method converges in just 1 iteration, preventing observation of convergence behavior. To properly compare both algorithms, I use the wider interval $[-100, 100]$ with tolerances $\epsilon = \delta = 10^{-12}$. This generates enough iterations to compute meaningful convergence orders and demonstrates robustness when starting far from the root. The MVT condition is satisfied: $f(-100) \cdot f(100) < 0$.

Convergence Analysis Results

Ridders' Method Ridders' method converged in **12 iterations** to machine precision ($\approx 10^{-16}$), as shown in Figures 5 and 6.

CONVERGENCE TABLE						
Itr	x3	f(x3)	Interval	x0	x2	d
1	-0.00000000	1.000e+00	2.000e+02	-100.00000000	100.00000000	2.000e+02
2	-0.03995206	9.592e-01	1.000e+02	-100.00000000	-0.00000000	1.000e+02
3	-0.11603628	8.770e-01	4.996e+01	-50.00000000	-0.03995206	4.996e+01
5	-0.44716888	4.395e-01	1.232e+01	-12.56800616	-0.25081132	1.232e+01
5	-0.44716888	4.395e-01	1.232e+01	-12.56800616	-0.25081132	1.232e+01
6	-0.62564127	1.435e-01	5.962e+00	-6.40940874	-0.44716888	5.962e+00
7	-0.69468974	1.664e-02	2.803e+00	-3.42828881	-0.62564127	2.803e+00
8	-0.70318157	5.436e-04	1.332e+00	-2.02696504	-0.69468974	1.332e+00
9	-0.70346505	4.511e-06	6.576e-01	-1.36082739	-0.70318157	6.576e-01
10	-0.70346742	9.127e-09	3.285e-01	-1.03200448	-0.70346505	3.285e-01
11	-0.70346742	4.489e-12	1.643e-01	-0.86773477	-0.70346742	1.643e-01
12	-0.70346742	5.551e-16	8.213e-02	-0.78560109	-0.70346742	8.213e-02

Figure 5: Ridders' method terminal output: 12 iterations to convergence.

```
=====
CONVERGENCE ORDER ANALYSIS:  $|e_{n+1}| \approx C|e_n|^p$ 
Using  $x^* \approx -0.703467422498391$  as true root
Formula:  $p \approx \log(|e_{n+1}|/|e_n|) / \log(|e_n|/|e_{n-1}|)$ 
=====
```

Itr	$ e_n = x_n - x^* $	Order p
1	7.035e-01	-
2	6.635e-01	-
3	5.874e-01	2.083
4	4.527e-01	2.140
5	2.563e-01	2.182
6	7.783e-02	2.095
7	8.778e-03	1.831
8	2.859e-04	1.569
9	2.372e-06	1.399
10	4.799e-09	1.295
11	2.360e-12	1.228
12	0.000e+00	N/A

```
-----
Estimated convergence order p: 1.717 (from iterations 4-11)
Theoretical Ridders order: 1.839
Bisection order: 1.000

✓ Superlinear convergence confirmed! (p > 1.7)
```

Figure 6: Ridders' convergence trajectory table: rapid decrease in interval size demonstrates superlinear convergence.

Bisection Method Bisection required **47 iterations**—almost $4\times$ more than Ridders—to achieve similar accuracy, as shown in Figures 7 and 8.

CONVERGENCE TABLE				
Itr	c	f(c)	Interval	a
1	0.00000000	1.000e+00	2.000e+02	-100.00000000
2	-50.00000000	2.500e+03	1.000e+02	-100.00000000
3	-25.00000000	6.250e+02	5.000e+01	-50.00000000
4	-12.50000000	1.562e+02	2.500e+01	-25.00000000
5	-6.25000000	3.906e+01	1.250e+01	-12.50000000
6	-3.12500000	9.722e+00	6.250e+00	-6.25000000
7	-1.56250000	2.232e+00	3.125e+00	-3.12500000
8	-0.78125000	1.525e-01	1.562e+00	-1.56250000
9	-0.39062500	5.240e-01	7.812e-01	-0.78125000
10	-0.58593750	2.133e-01	3.906e-01	-0.78125000
11	-0.68359375	3.750e-02	1.953e-01	-0.78125000
12	-0.73242188	5.570e-02	9.766e-02	-0.78125000
13	-0.70800781	8.650e-03	4.883e-02	-0.73242188
14	-0.69580078	1.454e-02	2.441e-02	-0.70800781
15	-0.70190430	2.971e-03	1.221e-02	-0.70800781
...				
(Showing first 15 of 47 iterations)				

Figure 7: Bisection method terminal output: 47 iterations to convergence.

```

=====
CONVERGENCE ORDER ANALYSIS:  $|e_{n+1}| \approx C|e_n|^p$ 
Using  $c^* \approx -0.703467422498250$  as true root
Formula:  $p \approx \log(|e_{n+1}|/|e_n|) / \log(|e_n|/|e_{n-1}|)$ 
=====

```

Itr	$ e_n = c_n - c^* $	Order p
1	7.035e-01	-
2	4.930e+01	-
3	2.430e+01	-0.166
4	1.180e+01	1.021
5	5.547e+00	1.044
6	2.422e+00	1.098
7	8.590e-01	1.250
8	7.778e-02	2.318
9	3.128e-01	-0.579
10	1.175e-01	-0.703
11	1.987e-02	1.815
12	2.895e-02	-0.212
13	4.540e-03	-4.923
14	7.667e-03	-0.283
15	1.563e-03	-3.035

```

-----
Estimated convergence order p: 1.104 (from iterations 4-15)
Theoretical bisection order: 1.000

```

Figure 8: Bisection convergence trajectory table (first 15 of 47 iterations): standard error analysis doesn't quite capture any particular convergence behavior, which is why typically bisection's convergence is described as linear with $|e_{n+1}| = \frac{1}{2}|e_n|$ based on interval halving.

Convergence Order Analysis

I computed the convergence order using: $p \approx \frac{\log(|e_{n+1}|/|e_n|)}{\log(|e_n|/|e_{n-1}|)}$ where $e_n = |x_n - x^*|$.

Ridders' Method: $p \approx 1.717$ (iterations 4–11), close to theoretical $p = 1.839$. This confirms **superlinear convergence**.

Bisection Method: $p \approx 1.104$ (iterations 4–15), close to theoretical $p = 1.0$. This confirms **linear convergence**.

Note: Bisection's order is typically expressed as $|e_{n+1}| = \frac{1}{2}|e_n|$ (interval halving), but for direct comparison with Ridders, I used the same formula above.

Conclusions

- **Efficiency:** Ridders requires 12 iterations vs. bisection's 47 ($4\times$ faster) for tolerance 10^{-12}
- **Convergence:** Ridders' $p \approx 1.717$ (superlinear) vs. bisection's $p \approx 1.0$ (linear)
- **Robustness:** Both converged successfully from the wide interval $[-100, 100]$

Ridders' method is substantially more efficient than bisection, making it the preferred choice when function evaluations are not prohibitively expensive.

A Code Implementations

This appendix contains the complete codebase for the problems solved in this midterm. All implementations are written in Julia and developed from scratch without using built-in numerical solver shortcuts.

A.1 Problem 3: LU and Cholesky Factorization

The following figures show the Julia implementation for Problem 3.

```

23  ✓ function get_LU_factors(A)
24      n = size(A, 1)
25      L = zeros(n, n)
26      U = zeros(n, n)
27
28  ✓ for r = 1:n
29      L[r, r] = 1.0
30  ✓ for c = 1:n
31  ✓     if r > c # L values before diagonal need to be computed
32      L[r, c] = (A[r, c] - sum([L[r, k]*U[k, c] for k ∈
33  ✓         1:c-1]))/U[c, c]
34      elseif c ≥ r # U values after diagonal need to be
35          computed
36      U[r, c] = A[r, c] - sum([L[r, k]*U[k, c] for k ∈
37          1:r-1])
38      end
39      return L, U
40  end get_LU_factors

```

Figure 9: LU Factorization function: `get_LU_factors(A)` implements the Doolittle algorithm with unit lower triangular matrix $\mathbf{L}$.

```

55 function solve_Ly_equals_b_for_y(L, b)
56     n = length(b)
57     y = zeros(n)
58
59     # Forward substitution: solve from top to bottom
60     for i = 1:n
61         sum_val = 0.0
62         for j = 1:i-1
63             sum_val += L[i,j] * y[j]
64         end
65         y[i] = (b[i] - sum_val) / L[i,i]
66     end
67
68     return y
69 end

```

Figure 10: Forward substitution function: `solve_Ly_equals_b_for_y(L, b)` solves the lower triangular system $\mathbf{Ly} = \mathbf{b}$ using simple nested loops.

```

84 ✓ function solve_Ux_equals_y_for_x(U, y)
85     n = length(y)
86     x = zeros(n)
87
88     # Backward substitution: solve from bottom to top
89 ✓     for i = n:-1:1
90         sum_val = 0.0
91 ✓         for j = i+1:n
92             sum_val += U[i,j] * x[j]
93         end
94         x[i] = (y[i] - sum_val) / U[i,i]
95     end
96
97     return x
98 end

```

Figure 11: Backward substitution function: `solve_Ux_equals_y_for_x(U, y)` solves the upper triangular system $\mathbf{Ux} = \mathbf{y}$ using simple nested loops.

```
145 function get_cholesky_factor(A)
146     n = size(A, 1)
147     L = zeros(n, n)
148
149     for i = 1:n
150         # Diagonal elements
151         sum_val = 0.0
152         for k = 1:i-1
153             sum_val += L[i,k]^2
154         end
155         L[i,i] = sqrt(A[i,i] - sum_val)
156
157         # Off-diagonal elements (below diagonal)
158         for j = i+1:n
159             sum_val = 0.0
160             for k = 1:i-1
161                 sum_val += L[j,k] * L[i,k]
162             end
163             L[j,i] = (A[j,i] - sum_val) / L[i,i]
164         end
165     end
166
167     return L
168 end
```

Figure 12: Cholesky factorization function: `get_cholesky_factor(A)` implements the standard Cholesky algorithm for symmetric positive-definite matrices, computing $\mathbf{L}$ such that $\mathbf{A} = \mathbf{L}\mathbf{L}^T$.

```

184 function solve_linear_system_cholesky(A, b)
185     # Step 1: Get Cholesky factor L where A = L*L'
186     L = get_cholesky_factor(A)
187
188     # Step 2: Solve Ly = b for y (forward substitution)
189     n = length(b)
190     y = zeros(n)
191     for i = 1:n
192         sum_val = 0.0
193         for j = 1:i-1
194             sum_val += L[i,j] * y[j]
195         end
196         y[i] = (b[i] - sum_val) / L[i,i]
197     end
198
199     # Step 3: Solve L'x = y for x (backward substitution with L')
200     x = zeros(n)
201     for i = n:-1:1
202         sum_val = 0.0
203         for j = i+1:n
204             sum_val += L[j,i] * x[j] # L' means we use L[j,i]
                                     # instead of L[i,j]
205         end
206         x[i] = (y[i] - sum_val) / L[i,i]
207     end
208
209     return x, L
210 end

```

Figure 13: Complete Cholesky solver: `solve_linear_system_cholesky(A, b)` combines Cholesky factorization with forward and backward substitution to solve $\mathbf{Ax} = \mathbf{b}$.

A.2 Problem 5-3: Ridders' Method Function

The following figures show the implementation of the Ridders' root-finding function, broken into logical parts. The bisection method is not shown here, as it was implemented in Homework 1 and is a subset of Ridders' approach.

```
132 ∨ function ridders_search(pr; verbose=false)
133     @unpack a, b, M, ε, δ, f = pr;
134     itr = 1
135     shouldStop = false
136
137     # Initial bracketing interval [x0, x2]
138     x0 = a
139     x2 = b
140     f0 = f(x0)
141     f2 = f(x2)
142
143     sgn_f0 = sign(f0)
144     sgn_f2 = sign(f2)
145
146 ∨ if sgn_f0 * sgn_f2 > 0
147     shouldStop = true
148     error("f(x0) and f(x2) must have opposite signs for
149         ridders_search to work.")
149     return nothing, nothing
150 end
```

Figure 14: Ridders' function (part 1): Initial setup and input checks.

```

152     while !shouldStop
153         # Compute midpoint x1
154         x1 = (x0 + x2) / 2
155         f1 = f(x1)
156
157         # Compute distance d
158         d = x2 - x0
159         You, 21 hours ago • huh? 🤖
160         myprintln(verbose, "itr=$itr, x0=$x0, x2=$x2, x1=$x1, f0=$f0,
161             f1=$f1, f2=$f2, d=$d")
162
163         # Check convergence on interval size
164         if abs(d) < δ
165             shouldStop = true
166             @error("Interval sufficiently small: |x2 - x0| = $d < δ =
167                 $δ")
168             return nothing, nothing
169         end
170
171         # Compute parameter a for h(x) = f(x)*e^(a*x)
172         # a = (2/d) * ln((f1 + sgn(f2)*sqrt(f1^2 - f2*f0)) / f2)
173         discriminant = f1^2 - f2 * f0
174
175         if discriminant < 0
176             shouldStop = true
177             @error("Negative discriminant: f1^2 - f2*f0 = $discriminant
178                 < 0")
179             return nothing, nothing
180         end
181     end

```

Figure 15: Ridders' function (part 2): Main loop and interval update.

```

169     # Compute parameter a for  $h(x) = f(x) * e^{(a*x)}$ 
170     #  $a = (2/d) * \ln((f_1 + \text{sgn}(f_2) * \sqrt{f_1^2 - f_2 * f_0})) / f_2$ 
171     discriminant = f1^2 - f2 * f0
172
173     if discriminant < 0
174         shouldStop = true
175         @error("Negative discriminant:  $f_1^2 - f_2 * f_0 = \$discriminant$ 
176             < 0")
177         return nothing, nothing
178     end
179
180     a_param = (2 / d) * log((f1 + sign(f2) * sqrt(discriminant)) /
181         f2)
182
183     # Compute h(x) values
184     h0 = f0 * exp(a_param * x0)
185     h1 = f1 * exp(a_param * x1)
186     h2 = f2 * exp(a_param * x2)
187
188     myprintln(verbose, "    a_param=$a_param, h0=$h0, h1=$h1,
189         h2=$h2")
190
191     # Compute x3 (x-intercept of line through (x0, h0) and (x2, h2))
192     #  $x_3 = (x_1 * h_2 - x_2 * h_1) / (h_2 - h_1)$ 
193     # Alternative form:  $x_3 = x_1 - h_1 * (x_1 - x_2) / (h_1 - h_2)$ 
194     if abs(h2 - h1) < 1e-14
195         shouldStop = true
196         @error("h2 - h1 too small: cannot compute x3")
197         return nothing, nothing
198     end

```

Figure 16: Ridders' function (part 3): Compute midpoint and evaluate function.

```

197     x3 = (x1 * h2 - x2 * h1) / (h2 - h1)
198     f3 = f(x3)
199
200     myprintln(verbose, "    x3=$x3, f3=$f3")
201
202     # Check convergence on function value
203     if abs(f3) < ε
204         shouldStop = true
205         println("Root found at x3 = $x3 with f(x3) = $f3")
206         return x3, f3
207     end

```

Figure 17: Ridders' function (part 4): Ridders' formula and root estimate.

```

209     # Determine next bracketing interval
210     sgn_f1 = sign(f1)
211     sgn_f3 = sign(f3)
212
213     if sgn_f1 * sgn_f3 < 0
214         # Root is in [x1, x3] or [x3, x1]
215         if x1 < x3
216             x0 = x1
217             x2 = x3
218             f0 = f1
219             f2 = f3
220         else
221             x0 = x3
222             x2 = x1
223             f0 = f3
224             f2 = f1
225         end
226     elseif sgn_f0 * sgn_f3 < 0
227         # Root is in [x0, x3]
228         x2 = x3
229         f2 = f3
230     elseif sgn_f2 * sgn_f3 < 0
231         # Root is in [x3, x2]
232         x0 = x3
233         f0 = f3
234     else
235         shouldStop = true
236         @error("Cannot determine next bracketing interval")
237         return nothing, nothing
238     end

```

Figure 18: Ridders' function (part 5): Interval selection and convergence check.

```

240     sgn_f0 = sign(f0)
241     sgn_f2 = sign(f2)
242
243     itr += 1
244     if itr > M
245         shouldStop = true
246         @error("Exceeded maximum iterations M = $M")
247         return nothing, nothing
248     end
249 end
250 end

```

You, 21 hours ago • huh? 🤖

Figure 19: Ridders' function (part 6): Return result and exit.